

Malware Analysis & Reverse Engineering

الكلية الجامعية
للعلوم التطبيقية
رائدة الإبداع



By: **Mahmoud Haitham Altawal**

ID: 120243939

Module 01 — Windows API & Structural Analysis

Task 1: API Analysis within Static Disassemblers (IDA Pro)

The disassembled sequence represents a call to the CreateFileA Windows API, configured to create/overwrite a hidden file named "psto.exe":

```
push 00000002h ; dwCreationDisposition = CREATE_ALWAYS
push 40000000h ; dwDesiredAccess = GENERIC_WRITE
push 00000000h ; dwShareMode = FILE_NO_SHARE
push 00000000h ; lpSecurityAttributes = NULL
push 00000002h ; dwFlagsAndAttributes = FILE_ATTRIBUTE_HIDDEN
push offset FileName ; "psto.exe"
call CreateFileA
```

Question A — CREATE_ALWAYS vs OPEN_EXISTING

CREATE_ALWAYS (0x2) instructs the API to always produce the target file: if the file already exists it is silently truncated and overwritten; if it does not exist, a new one is created. OPEN_EXISTING (0x3), in contrast, only succeeds if the target file is already present and fails outright otherwise.

Security implications when this is used by an injector or dropper:

- The dropper does not need to check for the file's presence beforehand — CREATE_ALWAYS guarantees the write succeeds in every scenario, which simplifies unattended payload deployment on any target machine.
- If a legitimate file already occupies that path/name, CREATE_ALWAYS will silently destroy and replace it — a classic file-replacement / binary-planting technique used to hijack a trusted filename or DLL search order.
- From a detection standpoint, CreateFileA/W calls using CREATE_ALWAYS against sensitive locations (System32, Startup folders, other processes' install directories) are a strong behavioral indicator, since legitimate installers typically use CREATE_NEW or OPEN_ALWAYS for executables rather than unconditionally overwriting them.

Question B — Why FileName is loaded as an offset

An x86 push instruction operand is a fixed-width value (a DWORD), so a variable-length string such as "psto.exe" cannot be embedded directly as an immediate operand. Instead:

- The string is stored as static data in the executable's .data or .rdata section at link/compile time.
- push offset FileName pushes the 32-bit memory address (pointer) of the first byte of that string onto the stack — not the string content itself.
- This matches the CreateFileA prototype, whose lpFileName parameter is declared as LPCSTR (a long pointer to a constant string), so the API always expects an address, never inline character data.
- For an analyst, this means the actual string must be located by following the offset into .rdata/.data — which is why malware authors frequently encrypt or dynamically resolve such strings at runtime to defeat static string-scanning tools.

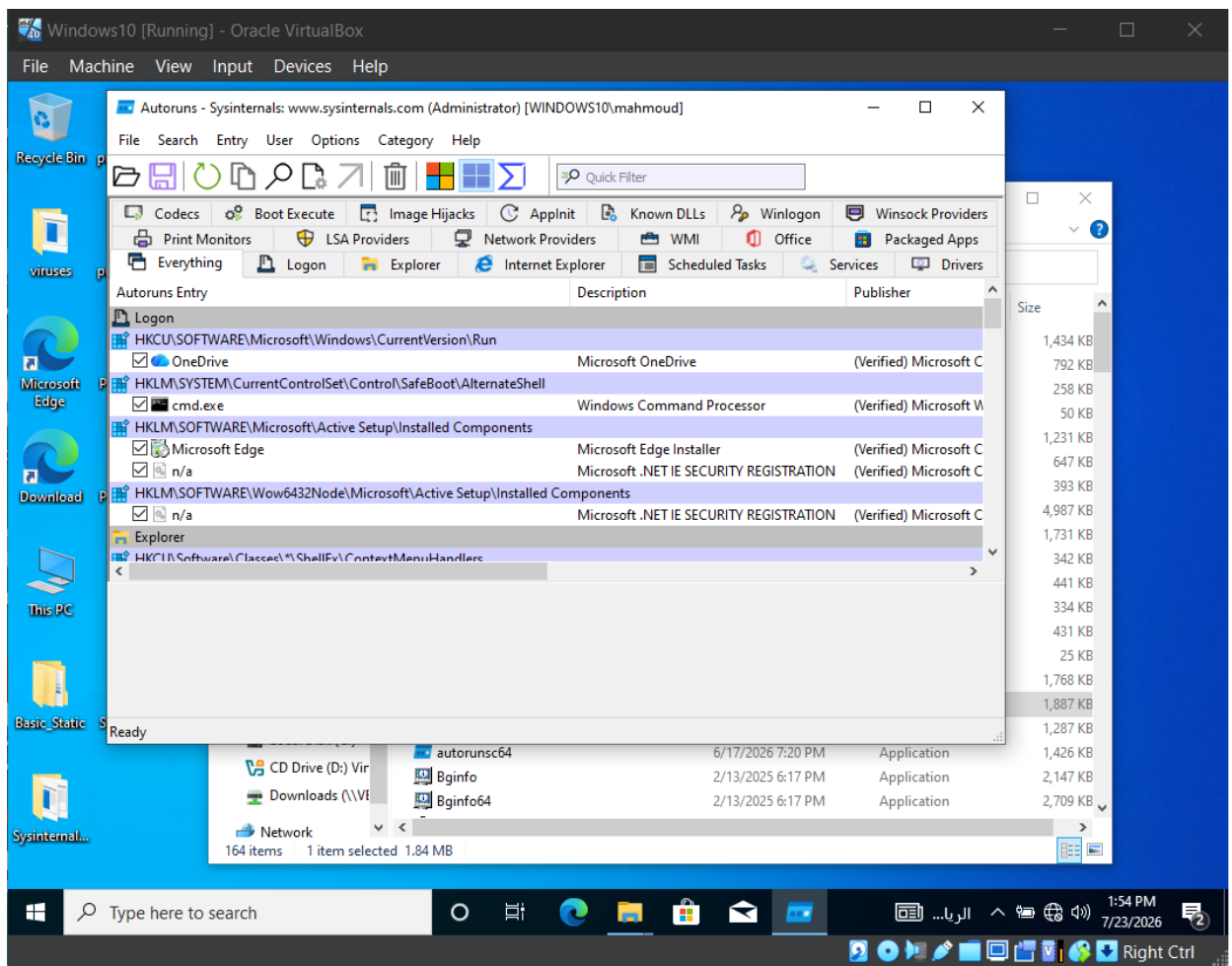
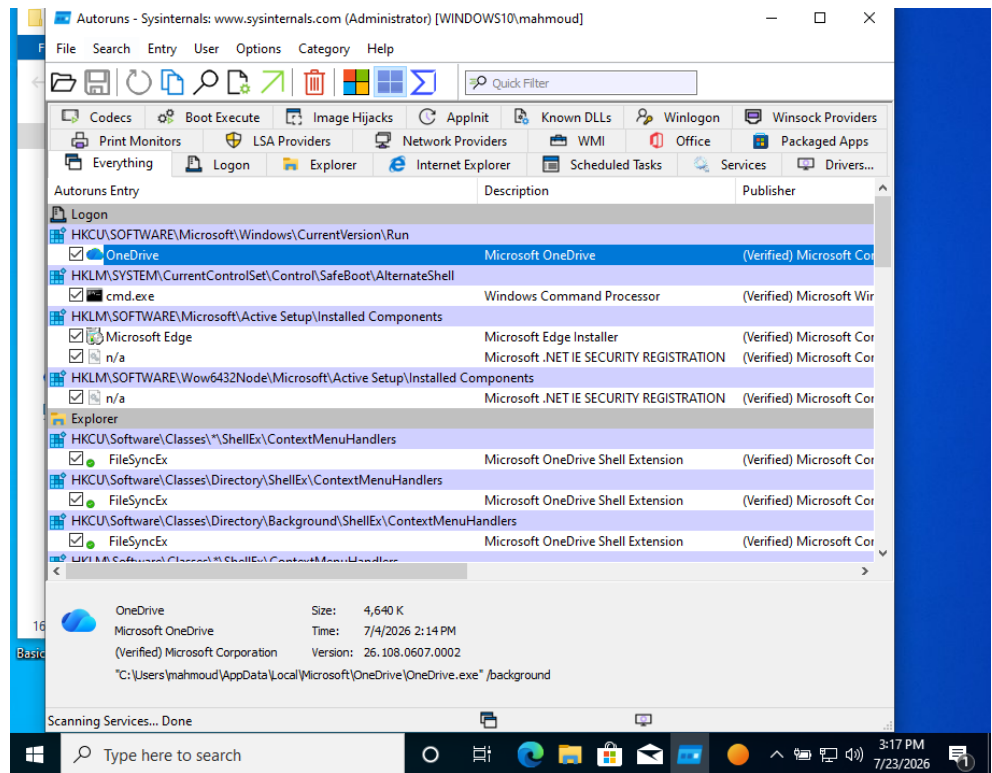
Module 02 — Registry Modification & Persistence Mechanisms

Task 2: Practical Lab Hunt Utilizing Sysinternals Autoruns

All practical steps below were performed inside an isolated Windows 10 VirtualBox sandbox, with the VM's network adapter disconnected (Not Attached) before any inspection of persistence entries, in line with safe malware-analysis practice.

Step 1 — Identifying a Persistent Entry Point

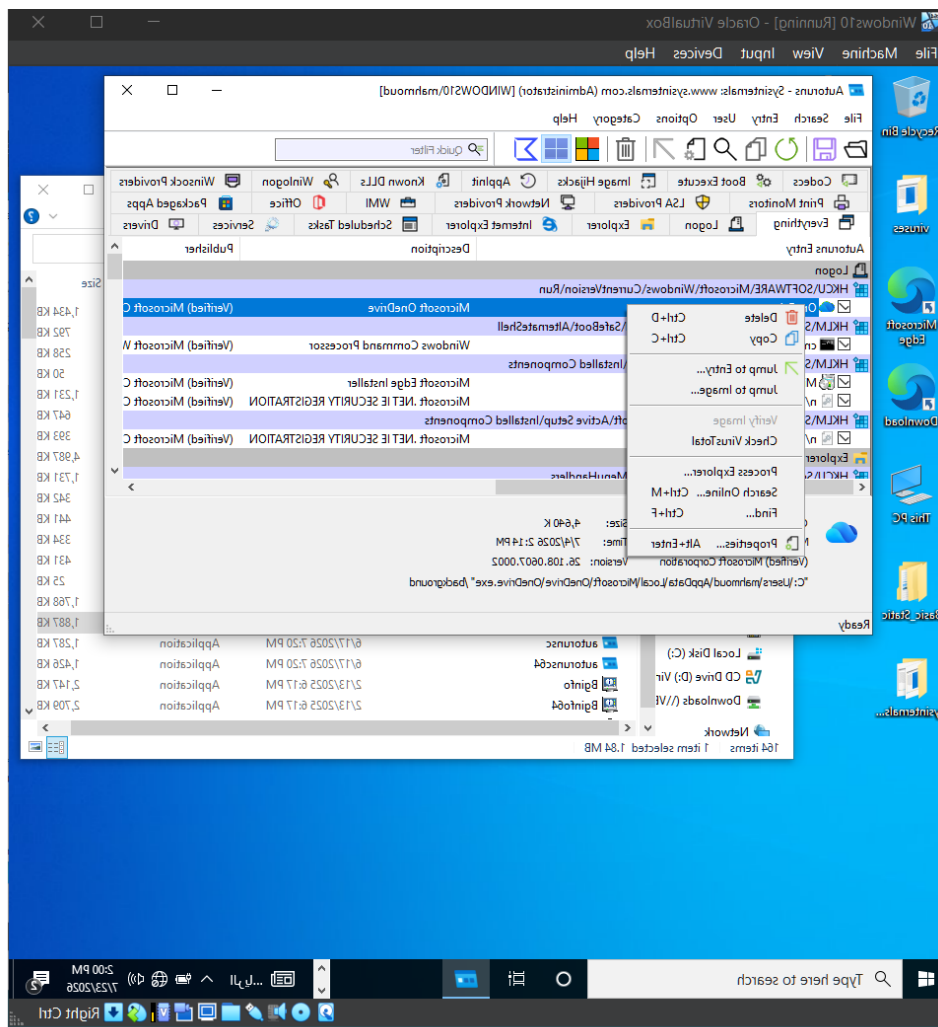
Sysinternals Autoruns (Autoruns64.exe) was downloaded and run as Administrator so that all Auto-Start Extension Points (ASEPs) across the system could be enumerated, not just the current user's.



Under the Logon tab, the following live entry was identified in the registry hive targeted by this exercise:

```
Hive : HKCU\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
Entry: OneDrive
Path : C:\Users\mahmoud\AppData\Local\Microsoft\OneDrive\OneDrive.exe /background
Publisher: (Verified) Microsoft Corporation
```

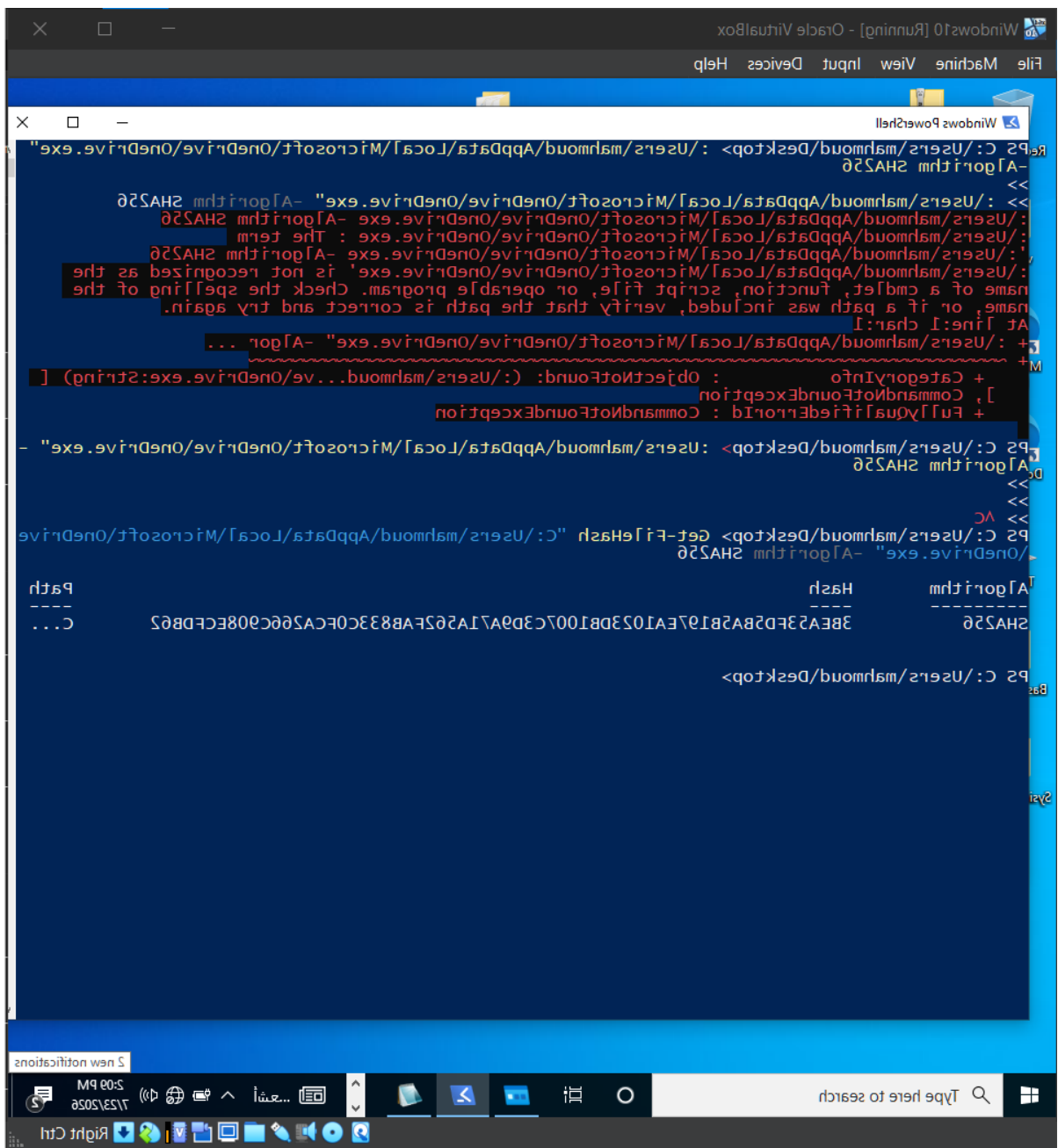
This entry was used as the case-study ASEP for this task. Although the binary itself is a legitimate, digitally-signed Microsoft component, the mechanism is identical to the one malware abuses: any executable dropped and referenced from this exact registry value will auto-launch at every user logon, giving it reboot-persistent execution — which is precisely the persistence primitive Module 02 describes.



Step 2 — Extracting the Cryptographic File Hash

The file path was obtained directly from the Autoruns entry details pane, then hashed locally using PowerShell's Get-FileHash cmdlet (run as Administrator):

```
Get-FileHash "C:\Users\mahmoud\AppData\Local\Microsoft\OneDrive\OneDrive.exe" -Algorithm
SHA256
```



Result:

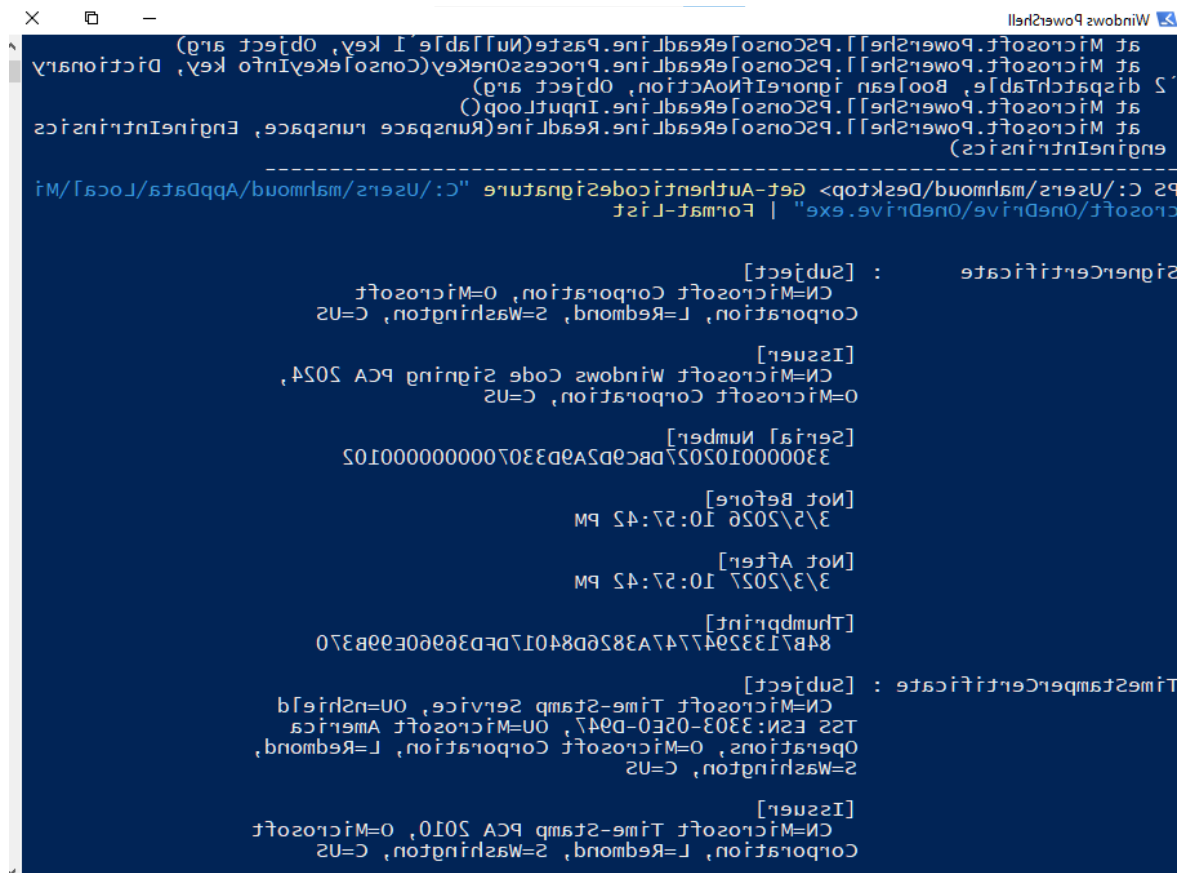
```
Algorithm : SHA256
Hash      : 3BEA53FD5BA5B197EA1023DB1007C3D9A71A562FAB833C0FCA266C908ECFDB62
File      : OneDrive.exe
```

Behavioral documentation: the binary launches automatically at user logon via the HKCU Run key, running in /background mode with no visible window — the same launch pattern (silent, background, Run-key-driven) that a persistence-focused implant would use to stay unnoticed. In a real incident this SHA-256 value would be submitted to VirusTotal / an internal threat-intel feed to confirm reputation before being trusted or quarantined.

Digital signature verification

As additional verification of the binary's authenticity (equivalent to Autoruns' "Verify Image" function), the Authenticode signature was checked directly via PowerShell:

```
Get-AuthenticodeSignature "C:\Users\mahmoud\AppData\Local\Microsoft\OneDrive\OneDrive.exe" |
Format-List
```



The result confirms Status: Valid and StatusMessage: Signature verified., with the certificate issued to CN=Microsoft Corporation by CN=Microsoft Windows Code Signing PCA 2024. This corroborates the "(Verified) Microsoft Corporation" publisher label shown earlier in Autoruns, giving a second, independent confirmation that the executable driving this ASEP has not been tampered with or substituted with an unsigned/malicious binary.

VirusTotal reputation check — attempted with the VM offline

As a final verification step, Autoruns' built-in Check VirusTotal option was invoked from the same right-click context menu on the ASEP entry, while the VM's network adapter remained disconnected (Not Attached) — consistent with safe, isolated malware-analysis practice.

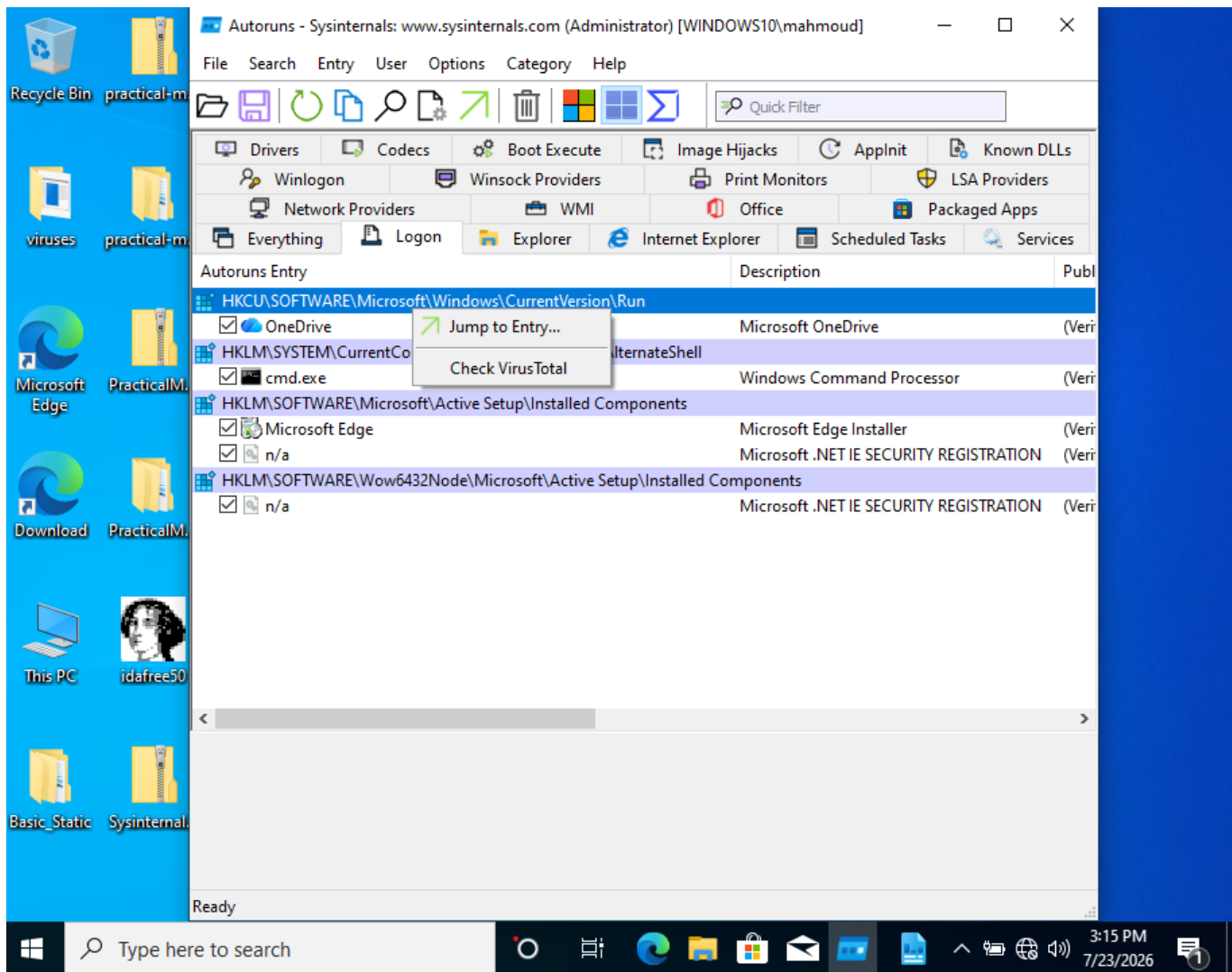


Figure 6 —.

No reputation data was returned and no request completed, because VirusTotal is a cloud lookup service that requires outbound internet access. This confirms the sandbox was genuinely network-isolated throughout the hands-on inspection, and illustrates why local techniques (Get-FileHash, Get-AuthenticodeSignature) were used as the primary evidence sources instead of relying on a live third-party reputation lookup.

Step 3 — Draft Operational Detection Rule

The assignment asks to draft (i.e., design/specify, not execute live) a rule capable of detecting unauthorized additions to the Run/RunOnce/Services ASEP hives. Two complementary approaches are provided below, as permitted by the task.

(a) Windows Event Log ID structure

Registry-key auditing is the correct primitive for catching new additions to a hive in real time (rather than inspecting a single file's static properties). This requires a SACL audit policy on the target keys plus monitoring of the corresponding Security event ID:

- Enable object access auditing: `auditpol /set /subcategory:"Registry" /success:enable /failure:enable`
- Apply a System Access Control List (SACL) for "Set Value" on each ASEP key: `HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run`, `\RunOnce`, `HKCU\...\Run`, and `HKLM\SYSTEM\CurrentControlSet\Services`.

- Monitor Event ID 4657 ("A registry value was modified") in the Security log — its EventData includes ObjectName (the full key path) and NewValue, enabling an automated alert whenever a new value is written under a monitored hive.
- Optionally pair this with Sysmon Event ID 13 (RegistryEvent – Value Set), which additionally reports the Image (originating process) that performed the write, making attribution to a specific dropper process straightforward.

(b) YARA rule

A complementary YARA rule can flag a dropped binary that is designed to establish this kind of persistence, by matching on the registry path strings and API imports it references:

```
rule Persistence_RunKey_Dropper
{
    meta:
        description = "Detects binaries referencing Run/RunOnce ASEP keys for persistence"
        author = "HW2 - Module 02"
        reference = "CurrentVersion\\Run, RunOnce, Services ASEPs"

    strings:
        $reg1 = "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run" ascii wide nocase
        $reg2 = "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\RunOnce" ascii wide nocase
        $reg3 = "CurrentControlSet\\Services" ascii wide nocase
        $api1 = "RegSetValueExA"
        $api2 = "RegSetValueExW"
        $api3 = "RegCreateKeyExA"

    condition:
        uint16(0) == 0x5A4D and          // valid PE (MZ header)
        1 of ($reg*) and 1 of ($api*)
}
```

Together, the YARA rule flags a suspicious binary at rest (static detection) while the Event Log / Sysmon rule catches the unauthorized registry write as it happens (runtime detection) — giving both prevention and detection coverage over the same persistence mechanism.

Module 03 — Process Control & Execution Transfer Mechanics

Task 3: Comparative Architectural Analysis — Win32 API vs. Native API (ntdll.dll)

Standard Win32 API calls (e.g., CreateFile, WriteProcessMemory, CreateRemoteThread from kernel32.dll) are thin wrappers that internally forward to their Native API counterparts inside ntdll.dll (e.g., NtCreateFile, NtWriteVirtualMemory, NtCreateThreadEx), which then execute a syscall instruction to transition into the kernel. Malware and offensive tooling increasingly call the ntdll.dll Native API functions directly, or even re-implement the syscall stub themselves, bypassing the Win32 layer entirely.

1. API hooking evasion against EDR

Modern EDR agents inject a DLL into every process and place inline hooks at the start of exported ntdll.dll functions, redirecting execution into the EDR's own inspection logic before the syscall executes. Attackers evade this in several ways:

- Direct syscalls: the malware extracts the syscall number for a given Native API function and issues the syscall instruction itself, skipping the hooked ntdll.dll function stub entirely, so the EDR's inline hook is never reached.
- Unhooking / stub restoration: the malware reads a clean copy of ntdll.dll from disk, or from a freshly mapped section, and overwrites the hooked in-memory bytes with the original unmodified instructions before calling the API.
- Indirect syscalls: the malware jumps into a legitimate, unhooked syscall instruction that already exists inside ntdll.dll (rather than embedding its own), which defeats EDR heuristics that flag syscalls executed from outside ntdll's memory region.

- Module stomping / manual mapping: the malicious code maps its own copy of ntdll.dll (or loads a whole payload manually) without going through the normal loader path that the EDR instruments, so no hook is ever installed for that instance.

2. Transition from Ring 3 (User Mode) to Ring 0 (Kernel Mode)

Regardless of which layer (Win32 or Native API) is used, the actual privilege transition happens at a single, fixed choke point:

- A Win32 call funnels down to its ntdll.dll Native API equivalent, which loads a syscall service number (SSN) into a register and executes the syscall (x64) / sysenter (legacy x86) instruction.
- This instruction causes the CPU to switch execution from Ring 3 to Ring 0, transferring control to the System Service Dispatcher (KiSystemCall64) inside ntoskrnl.exe.
- The dispatcher looks up the requested service number in the System Service Descriptor Table (SSDT) and jumps to the corresponding kernel-mode routine, which performs the privileged operation (file I/O, memory manipulation, thread creation, etc.) on the caller's behalf.
- Because this is the only doorway into kernel mode, it is also the layer where kernel-mode EDR components (minifilter drivers, ETWti, kernel callbacks) perform the deepest and hardest-to-evade inspection — user-mode hook evasion (point 1 above) does not defeat these kernel-level sensors, which is why direct/indirect syscalls only evade user-mode hooking EDRs, not kernel-mode telemetry.

Evidence & Environment Note

All Task 2 screenshots above were captured from a genuinely isolated VirtualBox Windows 10 VM with networking disabled during inspection, per safe malware-analysis practice. The ASEP entry examined (OneDrive.exe) is a legitimate, digitally-signed Microsoft binary used here purely as a live case study of the Run-key persistence mechanism described in Module 02; no live malicious sample was executed.